

Software de Gestión de la Configuración

CMS · Ansible

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 PI · Puesta en Producción de Aplicaciones

PI · PROYECTO INTERDISCIPLINAR

01

Infraestructura como Código

Orquestación, Configuración y DevOps

Infraestructura como Código

" *Gestionar y aprovisionar infraestructura mediante **ficheros de código**, en lugar de configurarla manualmente.*



SOFTWARE DE ORQUESTACIÓN

Crea escenarios completos: máquinas, redes, almacenamiento...

Terraform / OpenTofu



SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Configura el **software** de las máquinas ya creadas.

Puppet, Chef, Salt, Ansible

DevOps

Conflicto histórico: los equipos de Desarrollo (Dev) y de Sistemas (Ops) han tenido objetivos y herramientas distintas.

¿CÓMO LO SOLUCIONA DEVOPS?

- Mismas herramientas para dev y ops
- Extiende las buenas prácticas de desarrollo (Git, revisión de código, tests) a los sistemas
- Integración → entrega → despliegue **continuos**

¿TIENE RELACIÓN CON IAC?

Sí: la IaC es una de las piedras angulares de la cultura DevOps. Orquestación (OpenTofu) y configuración (Ansible) son las dos herramientas que la hacen posible.

PI · PROYECTO INTERDISCIPLINAR

02

Gestión de la Configuración

Configurar el software de las máquinas

¿Qué hace un software de gestión de la configuración?

Instala y configura el software de las máquinas de forma **automática, declarativa e idempotente**.

AUTOMATIZACIÓN

Instala y configura software **sin intervención manual** en cada máquina

DECLARATIVO

Se declara **qué estado** debe tener la máquina, no los pasos para llegar a él

IDEMPOTENCIA

Ejecutar la misma configuración **varias veces** da siempre el mismo resultado



Ejemplos: Ansible, Puppet, Chef, Salt.

PI · PROYECTO INTERDISCIPLINAR

03

Introducción a Ansible

Qué es, instalación y primeros pasos

¿Qué es Ansible?

" ***Ansible*** es una herramienta de **gestión de configuración y automatización** que permite definir, en ficheros de texto **YAML**, el estado deseado de servidores y aplicaciones, y aplicarlo **sin instalar agentes** en los nodos (usa **SSH**).

CARACTERÍSTICAS

- Desarrollado principalmente por **Red Hat**
- Escrito en **Python** · disponible en PyPI
- Arquitectura **push** sin agentes — solo SSH
- Configuración declarativa en **YAML**
- Primera versión: **2012**

¿Para qué sirve Ansible?

¿PARA QUÉ SIRVE?

- Configurar servidores de forma automática
- Instalar y gestionar paquetes y servicios
- Desplegar aplicaciones
- Orquestar tareas en **múltiples máquinas** a la vez
- Gestionar actualizaciones del sistema

Instalación de Ansible

CON APT (DEBIAN/UBUNTU)

```
apt install ansible
```

Instala la versión empaquetada por la distribución.

CON PIP EN ENTORNO VIRTUAL (RECOMENDADO)

```
python3 -m venv ansible-env  
source ansible-env/bin/activate  
pip install ansible
```

Permite tener la **versión más reciente** e independiente del sistema.



Ansible solo necesita instalarse en el **nodo de control** (tu máquina). Los nodos gestionados únicamente requieren Python y acceso SSH.

El inventario

El **inventario** es el fichero donde definimos los equipos que Ansible va a gestionar. Los equipos se organizan en **grupos**.

```
all:
  children:
    servidores:
      hosts:
        nodo1:
          ansible_ssh_host: 192.168.1.10
          ansible_ssh_user: usuario
          ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

- `all` — grupo raíz que engloba todos los equipos
- `servidores` — grupo personalizado con nuestros hosts
- Cada host puede tener variables propias: IP, usuario SSH, clave privada...

Fichero de configuración

Junto al inventario se crea un fichero `ansible.cfg` que configura el comportamiento de Ansible para el proyecto:

```
[defaults]
inventory = hosts
host_key_checking = False
```

- `inventory` — ruta al fichero de inventario
- `host_key_checking = False` — evita la verificación de la clave del host en cada conexión SSH (útil en entornos de pruebas)



Ansible busca este fichero en el directorio de trabajo. Cada proyecto puede tener su propia configuración.

Módulos y comandos ad-hoc

Un **módulo** permite ejecutar una acción concreta en uno o varios servidores remotos.

Los **comandos ad-hoc** ejecutan un módulo directamente desde la línea de comandos, sin necesidad de un playbook:

```
ansible <hosts> -m <módulo> -a "<parámetros>"
```

PARTE	DESCRIPCIÓN
<hosts>	all , nombre de grupo o nombre de host del inventario
-m <módulo>	módulo a usar
-a "<parámetros>"	argumentos del módulo
--become	ejecutar como root en el nodo remoto

Módulos esenciales (I)

PING

Comprueba la conectividad con los nodos. No es un ping ICMP, sino una verificación de conexión SSH y Python.

```
ansible all -m ping
```

Módulos esenciales (II)

COMMAND / SHELL

Ejecuta comandos en el nodo remoto. `shell` permite pipes, redirecciones y variables de entorno.

```
ansible all -m shell -a "echo $HOME | wc -c"
```

Módulos esenciales (III)

COPY

Copia ficheros desde el nodo de control al nodo remoto.

- `src` — fichero origen (local)
- `dest` — ruta de destino (remoto)
- `mode` — permisos (opcional)

```
ansible all -m copy -a "src=./index.html dest=/tmp/index.html mode=0644"
```

Módulos esenciales (IV)

APT

Instala, actualiza o elimina paquetes en sistemas Debian/Ubuntu.

- `name` — paquete
- `state` — `present`, `absent`, `latest`

```
ansible nodo1 -m apt -a "name=apache2 state=present" --become
```

Módulos esenciales (V)

SERVICE

Gestiona servicios del sistema.

- `name` — servicio
- `state` — `started`, `stopped`, `restarted`
- `enabled` — arranque automático

```
ansible nodo1 -m service -a "name=apache2 state=started enabled=yes" --become
```

Módulos esenciales (VI)

FILE

Gestiona ficheros, directorios y permisos.

- `path` — ruta en el nodo remoto
- `state` — `file`, `directory`, `absent`, `link`
- `mode` — permisos

```
ansible all -m file -a "path=/tmp/demo state=directory mode=0755"
```

Módulos esenciales (VII)

USER

Crea, modifica o elimina usuarios.

- `name` — nombre del usuario
- `state` — `present`, `absent`
- `shell`, `groups`

```
ansible all -m user -a "name=demo shell=/bin/bash state=present" --become
```

Declarativo e idempotencia

ENFOQUE DECLARATIVO

Ansible **no** usa un esquema imperativo (*"instala apache"*).

Declaramos el **estado deseado** del servidor:

*"Quiero que el servidor tenga
apache2 instalado"*

Ansible hará todas las operaciones necesarias para que ese estado se cumpla.

IDEMPOTENCIA

Si el estado declarado **ya se ha alcanzado**, Ansible no ejecuta ninguna operación adicional.

- Primera ejecución: instala apache → salida en **amarillo** (`changed`)
- Segunda ejecución: ya está instalado → salida en **verde** (`ok`)

Podemos ejecutar el mismo playbook **múltiples veces** con total seguridad.

PI · PROYECTO INTERDISCIPLINAR

04

Playbooks

Automatización declarativa con Ansible

Play y Playbook

PLAY

Una **jugada** es la unidad mínima de trabajo: declara **en qué hosts** se ejecuta y **qué tareas** se realizan.

Cada tarea usa un módulo con sus parámetros para alcanzar un estado deseado.

PLAYBOOK

Un **libro de jugadas** es un fichero YAML que agrupa uno o varios plays para conseguir una **configuración completa** de la infraestructura.

Se ejecuta con:

```
ansible-playbook site.yml
```

Estructura de un playbook

```
---
- name: Configurar servidor web          # Descripción del play
  hosts: servidores                     # A qué hosts se aplica
  become: true                          # Ejecutar tareas como root (sudo)

  tasks:

    - name: Actualizar caché de apt
      ansible.builtin.apt:
        update_cache: true

# ... resto de la lista de tareas
```

Variables en Ansible

Ansible puede trabajar con variables obtenidas de distintas fuentes:

- **Nivel de nodo** — definidas en el inventario para un host concreto
- **Nivel de grupo** — ficheros en `group_vars/` para todos los hosts de un grupo
- **Gathering Facts** — recopiladas automáticamente por Ansible de cada nodo

Variables: nivel de nodo

NIVEL DE NODO

Variables definidas directamente en el inventario para un host concreto.

```
hosts:
  nodo1:
    ansible_ssh_host: 192.168.1.10
    ansible_ssh_user: jose
    http_port: 80
```

Variables: nivel de grupo

NIVEL DE GRUPO

Variables en `group_vars/` que se aplican a todos los hosts de un grupo.

```
mi-proyecto/  
└─ group_vars/  
    └─ all          # Globales  
    └─ servidores  # Del grupo
```

group_vars/all

CONTENIDO

```
---
usuario_app: deploy
puerto_web: 80
paquetes:
  - apache2
  - git
  - curl
```

Uso de group_vars en el playbook

USO EN EL PLAYBOOK

Se referencian con dobles llaves `{{ }}`:

```
- name: Instalar paquetes
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop: "{{ paquetes }}"
```

Gathering Facts

Al inicio de cada play, Ansible ejecuta automáticamente la tarea **Gather Facts** que recopila información del nodo remoto.

VARIABLES DISPONIBLES

VARIABLE	VALOR DE EJEMPLO
ansible_hostname	servidor1
ansible_distribution	Debian
ansible_distribution_version	12
ansible_default_ipv4.address	192.168.1.10
ansible_memtotal_mb	2048

Uso de las facts en plantillas y tareas

USO EN PLANTILLAS Y TAREAS

```
- name: Mensaje de bienvenida
  ansible.builtin.copy:
    content: >
      Servidor: {{ ansible_hostname }}
      Sistema: {{ ansible_distribution }}
              {{ ansible_distribution_version }}
    dest: /etc/motd
```

Ver las facts de un nodo

Se pueden consultar todas las facts recopiladas de un nodo con el módulo `setup`:

```
ansible nodo1 -m setup
ansible nodo1 -m setup -a "filter=ansible_distribution*"
```

El módulo template

El módulo **template** copia una plantilla **Jinja2** al nodo remoto sustituyendo las variables por sus valores.

Las plantillas se guardan en el directorio **templates/** con extensión **.j2**:

templates/index.j2:

```
<!DOCTYPE html>
<html>
<body>
  <h1>{{ ansible_hostname }}</h1>
  <p>Sistema: {{ ansible_distribution }}
    {{ ansible_distribution_version }}</p>
  <p>Gestionado por: {{ usuario_app }}</p>
</body>
</html>
```

TAREA EN EL PLAYBOOK

```
- name: Desplegar página de inicio
  ansible.builtin.template:
    src: templates/index.j2
    dest: /var/www/html/index.html
    mode: "0644"
```

Condicionales y bucles en Jinja2

Jinja2 permite también **condicionales** y **bucles** dentro de las plantillas:

```
{% if ansible_distribution == "Debian" %}  
Sistema compatible.  
{% endif %}  
  
{% for pkg in paquetes %}  
- {{ pkg }}  
{% endfor %}
```

Estructura de un proyecto Ansible

```
mi-proyecto/  
├─ ansible.cfg      # Configuración de Ansible  
├─ hosts            # Inventario  
├─ site.yml         # Playbook principal  
├─ files/           # Ficheros estáticos para copiar con copy  
│   └─ foo.txt  
├─ templates/       # Plantillas Jinja2 para el módulo template  
│   └─ index.j2  
└─ group_vars/      # Variables por grupo  
    ├─ all           # Variables globales  
    └─ servidores    # Variables del grupo servidores
```

 Ansible localiza automáticamente los directorios **files/**, **templates/** y **group_vars/** si están junto al playbook o en el directorio de trabajo.

Ejecutar un playbook e interpretar la salida

```
ansible-playbook site.yml
```

COLORES DE LA SALIDA

COLOR	ESTADO	SIGNIFICADO
 Verde	<code>ok</code>	Ya estaba en el estado deseado
 Amarillo	<code>changed</code>	Se ha realizado un cambio
 Rojo	<code>failed</code>	La tarea ha fallado
 Azul	<code>skipping</code>	Tarea omitida (condición)

Resumen final (PLAY RECAP)

```
PLAY RECAP *****  
nodo1 : ok=4   changed=2   unreachable=0  
        failed=0   skipped=0
```

- `ok` + `changed` — tareas ejecutadas
- Si `changed=0` en la segunda ejecución → **idempotencia** funcionando
- `unreachable` o `failed` → hay que revisar errores

PI · PROYECTO INTERDISCIPLINAR

05

Roles

Organización y reutilización en Ansible

¿Qué es un rol?

*" Un **rol** es una unidad de configuración reutilizable que agrupa todas las tareas, ficheros, plantillas y variables necesarias para configurar **un servicio concreto**.*

MOTIVACIÓN

En un playbook con muchas tareas todo acaba en un único fichero. Los roles permiten:

- **Separar** la configuración de cada servicio
- **Reutilizar** el mismo rol en distintos proyectos
- **Asignar roles** a grupos de hosts diferentes
- Mantener el código **ordenado y mantenible**

EJEMPLO TÍPICO

Un proyecto con dos servidores:

- **commons** → tareas comunes a todas las máquinas
- **apache2** → instala y configura el servidor web
- **mariadb** → instala y configura la base de datos

Estructura de un rol

Cada rol es un directorio dentro de `roles/` con subdirectorios predefinidos:

```
roles/  
└─ apache2/  
    └─ tasks/  
        └─ main.yml      # Lista de tareas del rol  
    └─ handlers/  
        └─ main.yml      # Handlers (reinicio de servicios, etc.)  
    └─ templates/  
        └─ vhost.conf.j2 # Plantillas Jinja2  
    └─ files/  
        └─ index.html     # Ficheros estáticos para copiar  
    └─ defaults/  
        └─ main.yml       # Variables por defecto del rol
```

 Solo es necesario crear los subdirectorios que se vayan a usar. Ansible los detecta automáticamente por nombre.

Usando roles en el playbook

El fichero `site.yml` asigna cada rol al grupo de hosts que corresponde:

```
---  
# Tareas comunes  
- name: Configuración común  
  hosts: all  
  become: true  
  roles:  
    - commons  
  
# Servidor web  
- name: Configurar servidor web  
  hosts: servidores_web  
  become: true  
  roles:  
    - apache2
```

```
# Servidor de base de datos  
- name: Configurar servidor de base de datos  
  hosts: servidores_bd  
  become: true  
  roles:  
    - mariadb
```

Handlers

*" Un **handler** es una tarea especial que **solo se ejecuta cuando es notificada por otra tarea**. Se ejecuta **una sola vez**, al finalizar todas las tareas del play.*

roles/apache2/tasks/main.yml

```
- name: Copiar configuración de Apache
  ansible.builtin.template:
    src: vhost.conf.j2
    dest: /etc/apache2/sites-available/000-default.conf
  notify: Reiniciar Apache
```

El parámetro `notify` indica qué handler activar si la tarea produce un cambio (`changed`). Si el fichero ya era igual, el handler **no se ejecuta**.

roles/apache2/handlers/main.yml

```
- name: Reiniciar Apache
  ansible.builtin.service:
    name: apache2
    state: restarted
```

Una tarea normal se ejecuta siempre; el handler **solo se dispara si hubo un cambio real**.

Bucles con loop

Para ejecutar una tarea sobre una lista de elementos se usa `loop`. El valor de cada iteración se referencia con `{{ item }}`.



FORMA ANTIGUA (OBSOLETA)

```
- name: Instalar paquetes
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  with_items:
    - apache2
    - git
    - curl
```

`with_items` está **deprecado** desde Ansible 2.5.

FORMA ACTUAL

```
- name: Instalar paquetes
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - apache2
    - git
    - curl
```

O más idiomático para `apt`, pasando la lista directamente:

El módulo lineinfile

Permite **modificar una línea concreta** de un fichero remoto sin sobrescribir el fichero completo. Muy útil para ajustar ficheros de configuración.

```
- name: Configurar bind-address en MariaDB
  ansible.builtin.lineinfile:
    path: /etc/mysql/mariadb.conf.d/50-server.cnf
    regexp: '^bind-address'          # Línea a buscar (expresión regular)
    line: 'bind-address = 0.0.0.0'  # Valor que debe quedar
  notify: Reiniciar MariaDB
```

Parámetros de lineinfile

PARÁMETRO	DESCRIPCIÓN
<code>path</code>	Fichero a modificar en el nodo remoto
<code>regexp</code>	Expresión regular para localizar la línea
<code>line</code>	Contenido que debe tener esa línea
<code>state</code>	<code>present</code> (asegurar que existe) / <code>absent</code> (eliminar)

Gestión de bases de datos

Los módulos para MariaDB/MySQL pertenecen a la colección `community.mysql`, que hay que instalar antes de usarlos:

```
ansible-galaxy collection install community.mysql
```

COMMUNITY.MYSQL.MYSQL_DB

Crea, elimina o importa bases de datos.

```
- name: Crear base de datos
  community.mysql.mysql_db:
    name: "{{ db_name }}"
    state: present
    login_unix_socket: /var/run/mysqld/mysqld.sock
```

community.mysql.mysql_user

Gestiona usuarios y sus privilegios.

```
- name: Crear usuario de la BD
  community.mysql.mysql_user:
    name: "{{ db_user }}"
    password: "{{ db_password }}"
    priv: "{{ db_name }}.*:ALL"
    host: "%"
    state: present
    login_unix_socket: /var/run/mysqld/mysqld.sock
```

Ansible Galaxy

"Ansible Galaxy es el repositorio oficial de roles y colecciones creados y compartidos por la comunidad."

```
# Buscar un rol
ansible-galaxy search apache

# Instalar un rol
ansible-galaxy install geerlingguy.apache

# Instalar una colección
ansible-galaxy collection install community.mysql

# Ver roles instalados
ansible-galaxy list
```

PI · PROYECTO INTERDISCIPLINAR

¡Gracias!

Software de Gestión de la Configuración · Ansible

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI